

小组成员

- # Just make it

```
(fighting@kali)-[~/桌面/os_labcode/lab1]
$ make qemu
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/driver/console.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img

OpenSBI v1.6

Platform Name      : riscv-virtio,qemu
Platform Features  : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswi
Platform Timer Device : aclint-mtimer @ 10000000Hz

Boot HART Priv Version : v1.12
Boot HART Base ISA      : rv64imafdc
Boot HART ISA Extensions : sstc,zicntr,zihpm,zicboz,zicbom,sdtrig,svadu
Boot HART PMP Count     : 16
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits : 54
Boot HART MHPM Info     : 16 (0x0007fff8)
Boot HART Debug Triggers : 2 triggers
Boot HART MIDELEG       : 0x0000000000001666
Boot HART MEDELEG       : 0x0000000000f4b509
```

发现没有 (THU.CST) `os is loading ...` 输出, 也就说明没有执行 `kern_init`

```

int kern_init(void) {
    extern char edata[], end[];
    // 这里声明的两个符号由链接器定义，分别指向.data段结束和.bss段结束
    memset(edata, 0, end - edata);
    // 清除.bss段：由于内核没有标准库，memset需要我们自己实现

    const char *message = "(THU.CST) os is loading ...\n";
    cprintf("%s\n\n", message); // cprintf是我们在ucore中自己实现的格式化输出函数

    while (1)
        ; // 进入无限循环
}

```

也就是没有执行 lab1/kern/init/entry.s 里面的 `tail kern_init`，OpenSBI并没有执行内核。

后发现是opensbi版本太高了，对于makefile中的最后一句，他只会加载到内存中，却不会自动从0x80200000之后开始执行代码。

```

.PHONY: qemu
qemu: $(UCOREIMG) $(SWAPIMG) $(SFSIMG)
# $(V)$(QEMU) -kernel $(UCOREIMG) -nographic
$(V)$(QEMU) \
    -machine virt \
    -nographic \
    -bios default \
    -device loader,file=$(UCOREIMG),addr=0x80200000

```

后由AI建议改为 `-kernel bin/ucore.img` 会自动加载默认地址0x80200000,然后自动执行，为了保持和指导书一致最终选择降级版本。

替换qemu中的opensbi为低版本，即可成功运行。

```

(fighting@kali) - [~/桌面/os_labcode/lab1]
$ make qemu

OpenSBI v0.6

      _ _ _ _ _
     / / / / /
    / / / / /
   / / / / /
  / / / / /
 / / / / /
/ / / / /

Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFHIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 120 KB
Runtime SBI Version  : 0.2

MIDELEG : 0x0000000000001666
MEDELEG : 0x00000000000b509
PMP0    : 0x0000000000000000-0x000000000001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
(THU.CST) os is loading ...

```

GDB调试

我们打开两个终端，分别输入 `make debug` 以及 `make dbg`

```
问题 输出 调试控制台 终端 窗口 1
(fighting@kali) - [~/桌面/os_labcode/lab1]
$ make debug

(fighting@kali) - [~/桌面/os_labcode/lab1]
$ make gdb
riscv64-unknown-elf-gdb \
-ex 'file bin/kernel' \
-ex 'set arch riscv:rv64' \
-ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 10.1.0-2020.12.7) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb)
```

连接成功后，GDB 显示：

```
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
```

说明 CPU 当前停在 **复位地址** `0x1000`，即 OpenSBI 固件入口。

此时CPU停在RISC-V的复位地址，准备执行第一条指令。接下来我们将完整跟踪从0x1000到内核启动的整个过程。

阶段1：复位地址0x1000的引导代码

1. 查看并单步执行0x1000处的指令

```
(gdb) x/10i 0x1000
=> 0x1000:      auipc    t0,0x0
      0x1004:      addi    a2,t0,40
      0x1008:      csrr    a0,mhartid
      0x100c:      ld      a1,32(t0)
      0x1010:      ld      t0,24(t0)
      0x1014:      jr      t0
```

```
(gdb) info registers pc
pc                0x1000    0x1000
```

逐条执行6条指令

```
(gdb) si
(gdb) info registers t0
t0                0x1000    4096
```

```
(gdb) si
(gdb) info registers a2
a2                0x1028    4136
```

```
(gdb) si
(gdb) info registers a0
a0                0x0       0
```

```
(gdb) si
(gdb) info registers a1
a1          0x87e00000      2279604224

(gdb) si
(gdb) info registers t0
t0          0x80000000      2147483648

(gdb) si
0x0000000080000000 in ?? ()
```

2. 指令分析

地址	指令	功能说明	寄存器变化
0x1000	<code>auipc t0,0x0</code>	获取当前PC作为基址	t0 = 0x1000
0x1004	<code>addi a2,t0,40</code>	计算数据结构地址	a2 = 0x1028
0x1008	<code>csrr a0,mhartid</code>	读取CPU核心ID	a0 = 0x0
0x100c	<code>ld a1,32(t0)</code>	加载设备树地址	a1 = 0x87e00000
0x1010	<code>ld t0,24(t0)</code>	加载OpenSBI入口地址	t0 = 0x80000000
0x1014	<code>jr t0</code>	跳转到OpenSBI	PC = 0x80000000

这6条指令完成了最基础的引导工作：设置基址寄存器、读取CPU ID、加载设备树地址和OpenSBI地址，最后跳转到OpenSBI固件。

阶段2：OpenSBI固件执行

执行 `jr t0` 后，程序跳转到0x80000000，进入OpenSBI固件：

```
(gdb) x/5i $pc
=> 0x80000000: add    s0,a0,zero
    0x80000004: add    s1,a1,zero
    0x80000008: add    s2,a2,zero
    0x8000000c: jal    ra,0x80000608
    0x80000010: add    a6,a0,zero
```

```
(gdb) x/5i $pc
=> 0x80000000: add    s0,a0,zero
    0x80000004: add    s1,a1,zero
    0x80000008: add    s2,a2,zero
    0x8000000c: jal    ra,0x80000608
    0x80000010: add    a6,a0,zero
```

OpenSBI会执行硬件初始化、解析设备树、将内核加载到0x80200000，然后跳转到内核入口。为避免单步跟踪大量代码，我们直接在内核入口设置断点。

阶段3：到达内核入口

1. 设置断点并继续执行

```
(gdb) b *0x80200000
Breakpoint 1 at 0x80200000: file kern/init/entry.S, line 7.

(gdb) c
Continuing.

Breakpoint 1, kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop

(gdb) x/10i $pc
=> 0x80200000 <kern_entry>:    auipc    sp,0x3
    0x80200004 <kern_entry+4>:    mv      sp,sp
    0x80200008 <kern_entry+8>:    j       0x8020000a <kern_init>
    0x8020000a <kern_init>:      auipc    a0,0x3
    ...

(gdb) disassemble kern_entry
Dump of assembler code for function kern_entry:
=> 0x0000000080200000 <+0>:    auipc    sp,0x3
    0x0000000080200004 <+4>:    mv      sp,sp
    0x0000000080200008 <+8>:    j       0x8020000a <kern_init>
End of assembler dump.
```

```
(gdb) b *0x80200000
Breakpoint 1 at 0x80200000: file kern/init/entry.S, line 7.
(gdb) c
Continuing.

Breakpoint 1, kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop
(gdb) x/10i $pc
=> 0x80200000 <kern_entry>:    auipc    sp,0x3
    0x80200004 <kern_entry+4>:    mv      sp,sp
    0x80200008 <kern_entry+8>:    j       0x8020000a <kern_init>
    0x8020000a <kern_init>:      auipc    a0,0x3
    0x8020000e <kern_init+4>:      addi    a0,a0,-2
    0x80200012 <kern_init+8>:      auipc    a2,0x3
    0x80200016 <kern_init+12>:     addi    a2,a2,-10
    0x8020001a <kern_init+16>:     addi    sp,sp,-16
    0x8020001c <kern_init+18>:     li      a1,0
    0x8020001e <kern_init+20>:     sub     a2,a2,a0
(gdb) info registers pc sp
pc             0x80200000      0x80200000 <kern_entry>
sp             0x8001dd30      0x8001dd30
(gdb) disassemble kern_entry
Dump of assembler code for function kern_entry:
=> 0x0000000080200000 <+0>:    auipc    sp,0x3
    0x0000000080200004 <+4>:    mv      sp,sp
    0x0000000080200008 <+8>:    j       0x8020000a <kern_init>
End of assembler dump.
```

成功到达内核入口！kern_entry只有两条指令：初始化栈指针和跳转到kern_init。

阶段4：执行kern_entry

1. 初始化栈指针

```
(gdb) info registers sp
sp                0x8001dd30      0x8001dd30

(gdb) si
(gdb) info registers sp
sp                0x80203000      0x80203000 <bootstacktop>
```

执行 `1a sp, bootstacktop` 后，栈指针从OpenSBI传来的值变为bootstacktop (0x80203000)，为调用C函数准备好了运行时栈。

2. 跳转到kern_init

```
(gdb) si
9          tail kern_init

(gdb) si
kern_init () at kern/init/init.c:8
8          memset(edata, 0, end - edata);

(gdb) info registers pc
pc         0x8020000a      0x8020000a <kern_init>
```

成功从汇编代码跳转到C代码，内核初始化正式开始。

阶段5：kern_init执行

1. 反汇编kern_init

```
(gdb) disassemble kern_init
Dump of assembler code for function kern_init:
0x000000008020000a <+0>:      auipc    a0,0x3
0x000000008020000e <+4>:      addi     a0,a0,-2 # 0x80203008
0x0000000080200012 <+8>:      auipc    a2,0x3
0x0000000080200016 <+12>:     addi     a2,a2,-10 # 0x80203008
0x000000008020001a <+16>:     addi     sp,sp,-16
0x000000008020001c <+18>:     li      a1,0
0x000000008020001e <+20>:     sub     a2,a2,a0
0x0000000080200020 <+22>:     sd      ra,8(sp)
0x0000000080200022 <+24>:     jal     ra,0x802004b6 <memset>
0x0000000080200026 <+28>:     auipc    a1,0x0
0x000000008020002a <+32>:     addi     a1,a1,1186 # 0x802004c8
0x000000008020002e <+36>:     auipc    a0,0x0
0x0000000080200032 <+40>:     addi     a0,a0,1210 # 0x802004e8
0x0000000080200036 <+44>:     jal     ra,0x80200056 <cprintf>
0x000000008020003a <+48>:     j       0x8020003a <kern_init+48>
End of assembler dump.
```

2. 继续执行

```
# 执行memset
(gdb) si
...
memset (s=0x80203008, c=c@entry=0 '\000', n=0) at libs/string.c:275

# 执行cprintf
(gdb) c
Continuing.
```

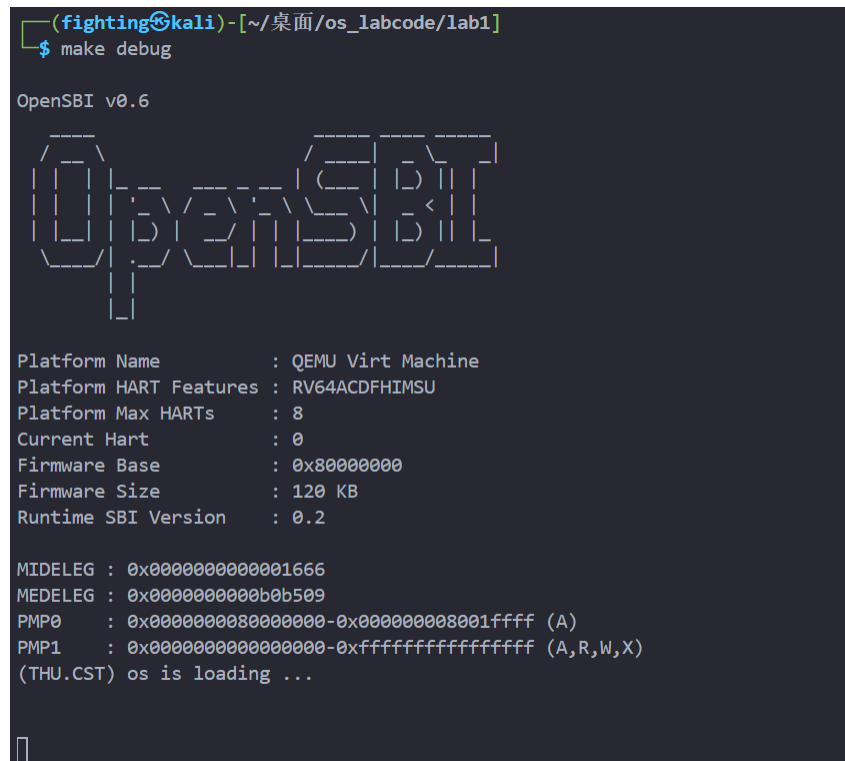
kern_init执行了三个主要操作：

1. 调用 `memset` 清零.bss段
2. 调用 `cprintf` 打印启动信息
3. 进入 `while(1)` 死循环

最终输出

在运行 `make debug` 的终端看到输出：

```
(THU.CST) os is loading ...
```



```
(fighting@kali)-[~/桌面/os_labcode/lab1]
$ make debug

OpenSBI v0.6

      _ _ _ _ _
     / /   / /
    / /   / /
   / /   / /
  / /   / /
 / /   / /
/_/_/_/_/_

Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFHIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 120 KB
Runtime SBI Version  : 0.2

MIDELEG : 0x0000000000001666
MEDELEG : 0x0000000000b0b509
PMP0    : 0x0000000080000000-0x000000008001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
(THU.CST) os is loading ...

█
```

启动流程总结



练习

练习1：理解内核启动中的程序入口操作

阅读 kern/init/entry.S内容代码，结合操作系统内核启动流程，说明指令 `la sp, bootstacktop` 完成了什么操作，目的是什么？ `tail kern_init` 完成了什么操作，目的是什么？

`la sp, bootstacktop`

- 操作说明：**

`la sp, bootstacktop` 是 RISC-V 汇编中的“Load Address”伪指令，用于将符号 `bootstacktop` 所代表的内存地址加载到栈指针寄存器 `sp` 中。

该符号在链接脚本中被定义为内核启动栈（`bootstack`）的**栈顶地址**，即栈空间的最高地址，通常位于 `.data` 段中通过 `.space KSTACKSIZE` 预留的一块静态内存区域的末尾。
- 目的：**

内核刚被 OpenSBI 跳转执行时，CPU 的栈指针处于未初始化状态。而后续调用 C 函数（如 `kern_init`）必须依赖有效的栈来保存返回地址、函数参数和局部变量。

因此，该指令的核心目的是**为内核初始化阶段建立一个可用的运行时栈**，确保 C 代码能够正常执行。

RISC-V 的栈向低地址增长，故 `sp` 必须初始化为栈空间的高地址（即 `bootstacktop`）。

tail kern_init

- 操作说明:

`tail kern_init` 是 RISC-V 汇编中的**尾调用跳转指令**（等价于 `j kern_init`），它直接将程序计数器（PC）设置为 `kern_init` 函数的入口地址，**并不保存返回地址**（即不修改 `ra` 寄存器）。

此时 `kern_init` 是用 C 语言编写的内核主初始化函数，其地址由链接器确定。

- 目的:

将控制权从汇编编写的**内核入口点**（`kern_entry`）正式移交到**C 语言实现的内核初始化逻辑**，完成从底层引导代码到高级语言运行环境的过渡。

In All

根据清华大学的ucore指导手册，一句话来说，entry干的事情很简单，"设置好 os 运行的堆栈（`la sp, boot_stack`语句。bootloader 并没有好心的设置好这些），然后调用 main 函数。main 函数位于 main.c 中，从此开始我们就基本进入了 C 的世界。",只不过我们这里不是 `call main`，而是 `tail kern_init`。但功能本质上是一样的。

练习2: 使用GDB验证启动流程

为了熟悉使用 QEMU 和 GDB 的调试方法，请使用 GDB 跟踪 QEMU 模拟的 RISC-V 从加电开始，直到执行内核第一条指令（跳转到 `0x80200000`）的整个过程。通过调试，请思考并回答：RISC-V 硬件加电后最初执行的几条指令位于什么地址？它们主要完成了哪些功能？请在报告中简要记录你的调试过程、观察结果和问题的答案。

答：位于地址 `0x1000`。

这是QEMU模拟的RISC-V的复位地址。CPU加电后，程序计数器PC被硬件初始化为`0x1000`，从这里开始执行ROM中的引导代码。

完成了哪些功能？

1. 建立地址基准 (`auipc t0,0x0`)

将当前PC值（`0x1000`）加载到`t0`寄存器，为后续相对地址访问提供基址。

2. 计算参数地址 (`addi a2,t0,40`)

计算`t0+40`得到`0x1028`，指向配置数据结构。

3. 获取CPU核心ID (`csrr a0,mhartid`)

读取当前CPU核心ID到`a0`（值为0），用于多核启动时区分不同核心。

4. 加载设备树地址 (`ld a1,32(t0)`)

从地址`0x1020`加载设备树（DTB）地址到`a1`（`0x87e00000`），设备树描述硬件配置。

5. 加载OpenSBI入口地址 (`ld t0,24(t0)`)

从地址`0x1018`加载OpenSBI固件入口地址到`t0`（`0x80000000`）。

6. 转交控制权 (`jr t0`)

跳转到OpenSBI固件，完成从ROM到固件的过渡。

详细流程可以看前面的GDB调试流程。

知识点总结与分析

一、实验中的重要知识点与OS原理的对应关系

1. 内存布局与链接 (Memory Layout & Linking)

实验中的体现：

- 链接脚本kernel.ld定义内核加载地址 (0x80200000)
- 符号edata、end标记数据段和BSS段边界
- bootstacktop定义栈顶地址

对应的OS原理知识点：

- 进程地址空间布局 (代码段、数据段、BSS段、栈、堆)
- 链接与加载过程

理解与分析：

- **含义：**链接脚本是实现原理中抽象内存布局概念的具体工具，它精确控制内核各段在物理内存中的位置。
- **关系：**原理课程讲解的是逻辑层面的地址空间划分，实验通过链接脚本和汇编代码将其具体化。
- **差异：**
 - 原理侧重概念和虚拟地址空间
 - 实验涉及物理地址分配和段的实际布局
 - 内核的内存布局比用户进程更加底层，需要考虑物理地址约束

2. 特权级与模式切换 (Privilege Levels)

实验中的体现：

- OpenSBI运行在M模式 (Machine Mode, 最高特权级)
- 内核运行在S模式 (Supervisor Mode)
- 通过SBI接口实现M模式到S模式的服务调用

对应的OS原理知识点：

- 用户态与内核态
- 系统调用机制
- 特权级保护

理解与分析：

- **含义：**RISC-V定义了M、S、U三个特权级，分别对应固件、操作系统内核、用户程序。
- **关系：**对应原理中的内核态和用户态概念，但RISC-V增加了M模式来处理最底层硬件操作。
- **差异：**
 - x86架构有Ring 0-3四个特权级，但实际只用Ring 0 (内核) 和Ring 3 (用户)
 - RISC-V的三级特权模型更清晰地分离了固件、OS和应用
 - SBI提供了标准化的跨特权级接口，类似于系统调用但位于更底层

3. 栈的初始化与管理 (Stack Management)

实验中的体现：

- `la sp, bootstacktop` 指令初始化栈指针
- 通过 `.space KSTACKSIZE` 静态分配内核栈
- 栈向低地址增长

对应的OS原理知识点：

- 栈的工作原理
- 函数调用约定 (Calling Convention)
- 栈帧结构

理解与分析：

- **含义：**栈是程序执行的基础设施，用于保存返回地址、参数、局部变量。
- **关系：**实验展示了从无栈环境到建立可用栈的过程，这是原理课程中默认已经存在的前提条件。
- **差异：**
 - 原理课程通常假设栈已经就绪，讲解栈的使用
 - 实验揭示了OS启动时必须手动初始化栈的事实
 - 内核栈和用户栈的管理策略不同（大小、分配方式）

4. 格式化输出与标准库实现 (cprintf & libc)

实验中的体现：

- 自实现 `cprintf`、`memset` 等函数
- 没有标准C库支持的裸机环境

对应的OS原理知识点：

- 系统调用与库函数的关系
- 内核与用户空间的边界

理解与分析：

- **含义：**内核不能依赖标准库，必须自己实现基础功能。
 - **关系：**揭示了通常使用的 `printf` 等函数背后的OS支持。
 - **差异：**
 - 用户程序通过系统调用使用OS提供的服务
 - 内核必须自给自足，实现自己的基础库
 - 内核 `printf` 的输出机制（如SBI串口输出）与用户态不同
-

• 二、OS原理中重要但实验未涉及的知识点

1. 进程管理 (Process Management)

- **原理内容**：进程的概念、进程控制块 (PCB)、进程状态转换、进程调度算法 (FCFS、RR、优先级调度等)
 - **未涉及原因**：Lab1仅关注系统启动，尚未建立进程抽象
 - **重要性**：进程是OS资源分配和调度的基本单位，是OS核心功能之一
-

2. 中断与异常处理 (Interrupt & Exception)

- **原理内容**：中断向量表、中断处理流程、异常分类 (缺页异常、非法指令等)
 - **部分涉及**：实验提到OpenSBI的中断初始化，但未深入展开
 - **重要性**：中断是OS响应外部事件的基础机制
-

3. 设备驱动 (Device Drivers)

- **原理内容**：I/O端口、DMA、设备控制器、驱动程序接口
 - **未涉及原因**：Lab1只使用了最基础的串口输出
 - **重要性**：驱动程序是OS与硬件交互的桥梁
-

4. 系统调用机制 (System Call)

- **原理内容**：用户态到内核态的切换、参数传递、系统调用表
 - **未涉及原因**：Lab1无用户程序，无需系统调用接口
 - **重要性**：用户程序获取OS服务的唯一途径
-

5. 设备树 (Device Tree, DTB)

- **原理内容**：设备树是一种数据结构，用来描述系统硬件信息 (CPU 核心数、内存大小与地址、外设位置和属性、中断控制器连接方式等)。它不是硬编码在内核中的，而是由固件 (如 OpenSBI 或 U-Boot) 传递给操作系统内核。
 - **未涉及原因**：Lab1 中虽然 OpenSBI 会传递设备树地址，但实验重点是验证内核的最小启动流程，没有深入解析设备树内容。
 - **重要性**：设备树让同一份操作系统内核能运行在不同硬件平台上，无需修改源码。它是嵌入式系统、RISC-V/ARM 平台适配硬件的重要机制。
-

6. 硬件线程 (Hart)

- **原理内容**：在 RISC-V 架构中，`hart` (hardware thread) 表示一个硬件执行上下文，即一个能够独立取指、执行的逻辑处理单元。每个 hart 有自己独立的程序计数器 (PC)、寄存器组，但可能共享一个核心的执行资源 (类似于 x86 中的超线程)。
- **未涉及原因**：Lab1 主要针对单核单 hart 环境进行实验，没有涉及多 hart 的启动与同步问题。

- **重要性：**在多核多线程的处理器上，每个 hart 都会从复位向量（0x1000）开始执行内核入口代码。操作系统需要根据 `mhartid` 区分不同的 hart，并负责启动、调度和管理它们。